Serving ML Models with FastAPI

This is one of the most important topics in Machine Learning Engineering because it answers a critical question:
"I trained my model. Now how can other people use it?"
Training a model is only half the job.
The real goal is:
Make the model accessible through an API so applications, websites, and users can send data and get predictions.

---

1. Introduction

Up to now you've learned:

- FastAPI basics
- HTTP Methods
- Path Parameters
- Query Parameters
- Pydantic Models
- Request Bodies

Now it's time to combine everything.
The objective:

ML Model
↓
FastAPI
↓
REST API
↓
Applications / Users

What Does "Serving a Model" Mean?
Many beginners think:
"I trained a model. My work is done."
Not really.
A trained model sitting in a notebook cannot help users.
Example:
You trained a sentiment analysis model:
Input:
"This movie is amazing"

Output:
Positive
Only you can use it locally.
To make it useful:

- Website sends text
- API receives text
- Model predicts
- API returns prediction

This process is called:
Model Serving

---

2. Plan of Attack

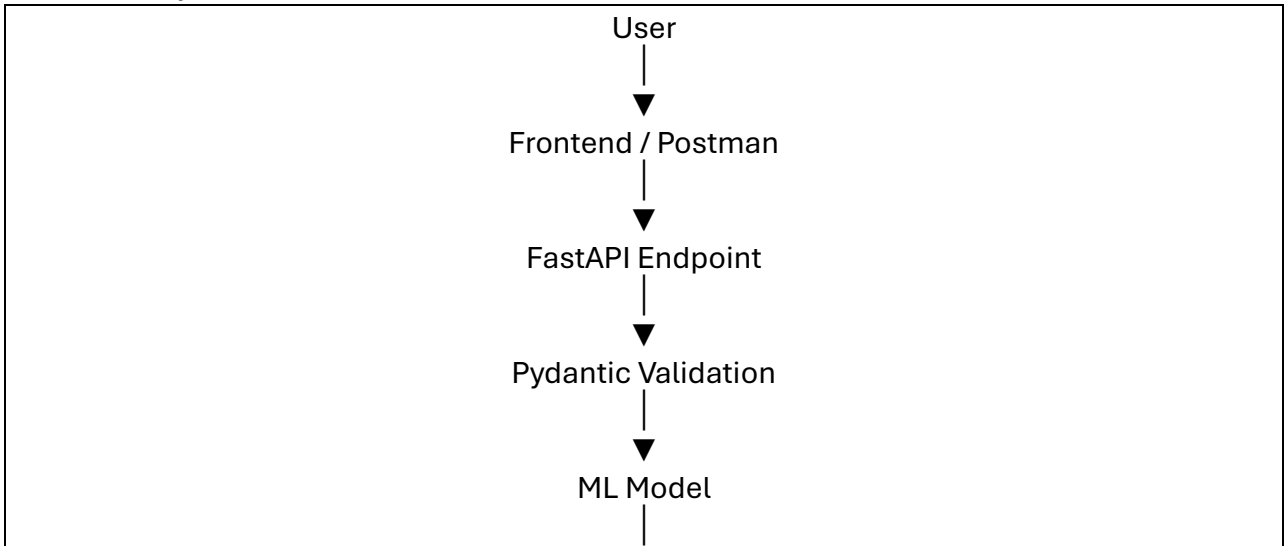
The typical workflow is:

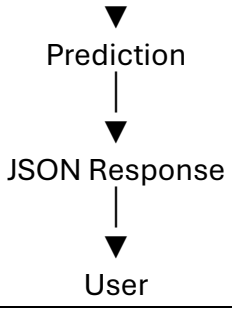
1. Collect Data
↓
2. Train Model
↓
3. Save Model
↓
4. Build FastAPI Application
↓
5. Load Saved Model
↓
6. Create Prediction Endpoint
↓
7. Receive User Input
↓
8. Generate Prediction
↓
9. Return Response

---

3. Flow Diagram

The entire system looks like this:





Understanding Each Component

User
Sends input.
Example:

```
{
  "age": 25,
  "salary": 50000
}
```

FastAPI Endpoint
Receives request.
Example:

```
@app.post("/predict")
```

Pydantic
Validates data.
Checks:

```
age: int
salary: float
```

ML Model
Runs prediction.
Example:

```
prediction = model.predict(...)
```

Response
Returns result.

```
{
  "prediction": "Approved"
}
```

4. Building and Exporting the Model
Before FastAPI can use a model, the model must be trained and saved.

Example: Simple Classification Model

Suppose we train:

```
from sklearn.linear_model import LogisticRegression
```

Training

```
model.fit(X_train, y_train)
```

After training:

```
model.predict(...)
```

works correctly.

Why Save the Model?

Without saving:

Every server restart requires retraining.

Very inefficient.

Save Model

Common methods:

Pickle

```
import pickle

pickle.dump(model, open("model.pkl", "wb"))
```

Joblib

```
import joblib

joblib.dump(model, "model.pkl")
```

Result

You get:

```
model.pkl
```

This file contains:

- Learned weights
- Parameters
- Training knowledge

Mental Model

Think of:

```
model.pkl
```

as a trained brain stored on disk.

5. **Code Demo (Loading Model)**

When FastAPI starts:

```
import joblib

model = joblib.load("model.pkl")
```

Why Load Once?

Good:

```
model = load_model()
```

loaded once when server starts.

Bad:

```
@app.post("/predict")
def predict():
   model = load_model()
```

Loads model for every request.

Very slow.

Best Practice

Load model globally during startup.

```
model = joblib.load("model.pkl")
```

---

6. Building the API Endpoint

Now we connect FastAPI with the model.

Step 1: Define Input Schema

Using Pydantic:

```
from pydantic import BaseModel

class PredictionInput(BaseModel):
   age: int
   salary: float
```

Why?

FastAPI now knows:

```
{
  "age": 25,
  "salary": 50000
}
```

is required.

Step 2: Create Endpoint

```
@app.post("/predict")
```

```
def predict(data: PredictionInput):
```

Step 3: Extract Features

```
features = [[
  data.age,
  data.salary
]]
```

Step 4: Predict

```
prediction = model.predict(features)
```

Step 5: Return Result

```
return {
  "prediction": prediction[0]
}
```

Complete Flow

```
@app.post("/predict")
def predict(data: PredictionInput):

  features = [[
    data.age,
    data.salary
  ]]

  prediction = model.predict(features)

  return {
    "prediction": prediction[0]
  }
```

7. Code Demo (Complete Project)

A simplified production structure:

```
project/
│
 ├── app.py
 ├── model.pkl
 ├── requirements.txt
```

app.py

```
from fastapi import FastAPI
from pydantic import BaseModel
```

```
import joblib

app = FastAPI()

model = joblib.load("model.pkl")

class InputData(BaseModel):
  age: int
  salary: float

@app.post("/predict")
def predict(data: InputData):

  features = [[
    data.age,
    data.salary
  ]]

  prediction = model.predict(features)

  return {
    "prediction": int(prediction[0])
  }
```

Running the API

```
uvicorn app:app --reload
```

Server starts:

```
http://127.0.0.1:8000
```

API Documentation

Open:

```
http://127.0.0.1:8000/docs
```

FastAPI automatically creates:

- Interactive documentation
- Request testing UI
- Validation rules

8. Testing the API

Testing ensures everything works.

Method 1: Swagger UI

Visit:

```
http://127.0.0.1:8000/docs
```

Click:

```
POST /predict
```

Enter:

```
{
  "age": 25,
  "salary": 50000
}
```

Press:
Execute

Response:

```
{
  "prediction": 1
}
```

Method 2: Postman

Send:

```
POST /predict
```

Body:

```
{
  "age": 25,
  "salary": 50000
}
```

Receive:

```
{
  "prediction": 1
}
```

Method 3: Python Requests

```
import requests

response = requests.post(
   "http://127.0.0.1:8000/predict",
   json={
      "age": 25,
      "salary": 50000
   }
```

```
)

print(response.json())
```

Real ML Examples

Sentiment Analysis

Input:

```
{
  "text": "This movie is fantastic"
}
```

Output:

```
{
  "sentiment": "Positive"
}
```

House Price Prediction

Input:

```
{
  "area": 2000,
  "bedrooms": 3
}
```

Output:

```
{
  "price": 250000
}
```

Disease Prediction

Input:

```
{
  "age": 45,
  "blood_pressure": 140
}
```

Output:

```
{
  "risk": "High"
}
```

Your Pashto ASR Project

Input:

Audio File

Flow:

```
Audio
↓
FastAPI
↓
Whisper Model
↓
Transcription
↓
JSON Response
Output:
{
"transcription": "..."
}
```

This is exactly how production speech-to-text systems work.

Common Beginner Mistakes

Loading Model Inside Endpoint

Bad:

```
@app.post("/predict")
def predict():
   model = joblib.load(...)
```

Loads model every request.

Load Once

```
model = joblib.load(...)
```

at startup.

No Validation

Using raw dictionaries:

```
def predict(data: dict):
```

Less safe.

Use Pydantic

```
class InputData(BaseModel):
```

Returning Numpy Objects

Bad:

```
return {
   "prediction": prediction
}
```

Can cause serialization issues.

Convert First

```
return {
   "prediction": int(prediction[0])
}
```

Complete Mental Model

Train Model
↓
Save Model (.pkl)
↓
Load Model in FastAPI
↓
Create Pydantic Input Schema
↓
Create /predict Endpoint
↓
Receive Request Body
↓
Validate Input
↓
Run Prediction
↓
Return JSON Response

Think of FastAPI as a receptionist:

User arrives
↓
Receptionist (FastAPI)
↓
Checks form (Pydantic)
↓
Consults expert (ML Model)
↓
Returns answer

The receptionist doesn't make decisions.
The ML model does.
FastAPI simply receives requests, sends them to the model, and returns the prediction.

Serving an ML model with FastAPI means:

1. Train the model.
2. Save it (.pkl).
3. Load it when FastAPI starts.
4. Create a Pydantic request schema.

5. Build a /predict endpoint.
6. Validate incoming data.
7. Run model.predict().
8. Return predictions as JSON.

This pattern is the foundation of almost every production ML API, including recommendation systems, chatbots, speech recognition systems, fraud detection systems, computer vision services, and your own Pashto speech-to-text application.